

Product Requirements Document & Technical Specifications

System: ISKCON Devotee Management System

****Document Version:**** 1.0.0

****Effective Date:**** June 6, 2026

****Status:**** Approved for Production

****Author:**** AI Coding & Product Engineering Team

1. Executive Summary & Vision

1.1 Context & Problem Statement

Temples and faith-based communities historically manage attendee outreach, weekly service registration, follow-up calls, and session attendance through highly fragmented channels (such as generic paper registries, decentralized Excel spreadsheets, and informal instant messaging groups). This fragmentation leads to:

- ****Disjointed outreach:**** Callers (facilitators) overlapping and dialing the same devotee, or conversely, devotees being forgotten.
- ****Data corruption & duplicates:**** High occurrence of misspelled names, varying phone number formats (`9876...` vs `+91 9876...`), and duplicate records.
- ****Lack of progression tracking:**** Impracticable tracking of devotee activity, spiritual progress (rounds of chanting), or historical attendance trends.
- ****Resource exhaustion:**** High network and UI compute overhead when trying to search, list, or update thousands of records on single-session clients.

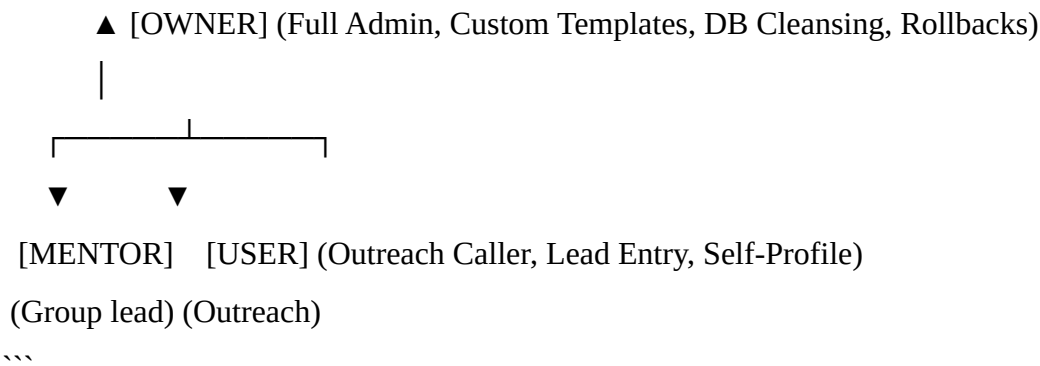
1.2 The Solution

The ****ISKCON Devotee Management System**** is a unified, real-time, mobile-first administrative dashboard and devotee CRM. The system bridges the gap between administrators (Owners) and local coordinators (Mentors and Facilitators). It securely segments administrative analytics, event planning, calling assignments, attendance logs, and audit histories into a highly cohesive, secure workspace.

2. User Personas & Role-Based Access Control (RBAC)

The application employs a rigid, three-tier hierarchical role permission pattern (`UserRole = 'USER' | 'MENTOR' | 'OWNER'`).

...



2.1 Role Capabilities Matrix

Feature Module	Guest / Public	User (Facilitator)	Mentor (Coordinator)	Owner (Administrator)
	:---	:---	:---	:---
Authentication & Self Profile	Read Only	Create / Update	Create / Update	Create / Update
Add Devotee Record	Denied	Allowed	Allowed	Allowed
Modify Devotee globally	Denied	Denied	Denied	Allowed
Delete Devotee globally	Denied	Denied	Denied	Allowed
View Global Devotee DB	Denied	Group Only (Filtered)	Group Only (Filtered)	Full Access (All Records)
Create/Edit/Delete Events	Denied	Denied	Denied	Allowed
View Event Assignments	Denied	Assigned Only	Assigned Only	Complete Access
Submit / Update Calling Logs	Denied	Assigned Only	Assigned Only	Complete Access
Manage Custom Templates	Denied	Denied	Denied	Allowed
Mark Event Attendance	Form Only	Form Only	Allowed	Complete Access
Undo / Redo History Logs	Denied	Denied	Denied	Complete Access

3. Product Features & Functional Requirements

3.1 Authentication & User Management

- **Security Check:** All endpoints (excluding public attendance links) are strictly protected behind an active Auth Guardian.
- **Dynamic Onboarding:** Unauthenticated users are routed to `/login``. Upon authentication, the system retrieves their `UserProfile`` from Firestore.
- **Initial Synchronization:** The system automatically locks the UI and presents an elegant "Syncing Soul" loading spinner while verified attributes and security claims are parsed.

3.2 Devotee CRM Directory

- **Profile Registration:** Captures rich metrics such as Name, Contact Number, Mentor Name, Facilitator, Age, Address, Gender, Institute/Workplace, DOB, and Chanting Progress (Number of Rounds).
- **History Logs & Notes:** Displays a persistent timeline of caller facilitation logs (`FacilitationNote[]``). Allows callers to append dated notes describing spiritual milestones or personal challenges during contact.
- **Soft Deletions:** Devotees must never be hard-deleted by mistake. Deletion operates as a logical flag (`isDeleted: true; deletedAt: ISOString``) accessible only by the `OWNER``.

3.3 Duplicate Detection & Cleansing Logic

To curb dirty data, the system runs strict validation checks on creation and CSV import:

1. **Case-Insensitive Exact Name Matching:** Normalizes names via `.toLowerCase().trim()`` to intercept duplicate entries (e.g., ``Amit`` vs ``amit``).
2. **Contact Number Standardization:** Reconciles phone numbers containing country codes, leading zeroes, dashes, or spacing (e.g., matching ``9810123456`` with ``+91 98101 23456``).
3. **Phonetic & Levenshtein Distance Safeguards (Heuristic):** Warns administrators when phonetically similar names (e.g., ``Amit`` vs ``Amith``, ``Prabh`` vs ``Prabhu``) are added.
4. **Duplicate Association Handling:** Flags suspect records with attributes `isDuplicate: true`` and `duplicateType: 'complete' | 'partial_name' | 'partial_contact``. Displays a dedicated "Database

Cleansing Dashboard" to the `OWNER` to merge or dismiss duplicates.

3.4 Dynamic Form Configurations (Templates)

- Allows administrative owners to define custom tracking templates for specific events.
- Rather than forcing a static attendee list layout, owners can dynamically include fields like `['name', 'contact', 'chanting', 'institute', 'address']`.
- Custom templates are stored in an independent `/templates` collection and can be associated with any planned `Event` by its unique identifier `templateId`.

3.5 calling Outreach Assignments Engine

- **Strategic Dispatch:** An `OWNER` couples devotees to facilitators (`CallingAssignment`) for an upcoming festival or sermon.
- **Caller Workspace:** Facilitators log into their personalized user dashboard and view only their calling assignments.
- **Status & Response Caching:** The caller contacts the devotee and updates status (`PENDING` -> `COMPLETED`) alongside responses:
 - `COMING`: Devotee confirmed attendance.
 - `NOT_COMING`: Devotee declined.
 - `MAYBE`: Uncertain; requires follow-up.
 - `UNREACHABLE`: Left voicemail, disconnected, or no answer.
 - `NONE`: No contact made.
- Facilitators can write custom feedback strings (`responseText`). All state transfers automatically update timestamps.

3.6 Attendance Tracking & Marking

- **Dual Flow Execution:**
 - **Internal Admin Flow:** Owners and Mentors mark attendance by searching devotees and checking them off in real-time.
 - **Public Attendance Flow:** Public forms (`/public-attendance/:id`) allow devotees to register themselves at the temple entry by scanning a QR code pointing directly to the event URL.
- **Attendance Open/Close Safeguard:** Owners can toggle `isAttendanceOpen` on an event. Once disabled, any further update requests to that event's attendance subcollection are automatically blocked.

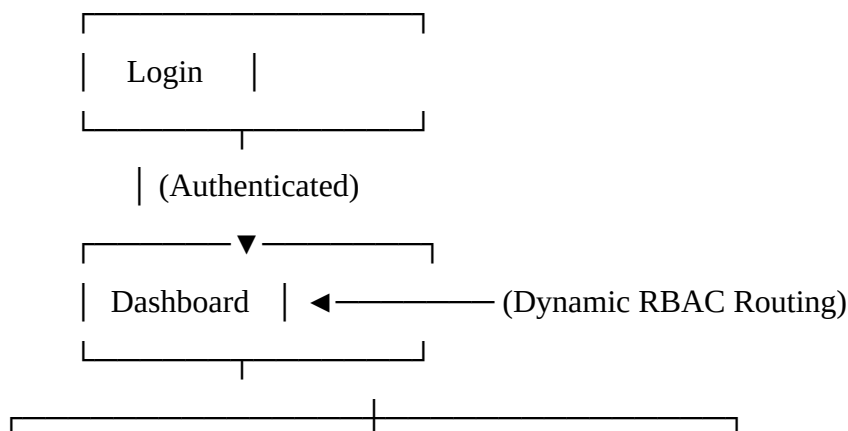
4. Technical Architecture & Structural Schemas

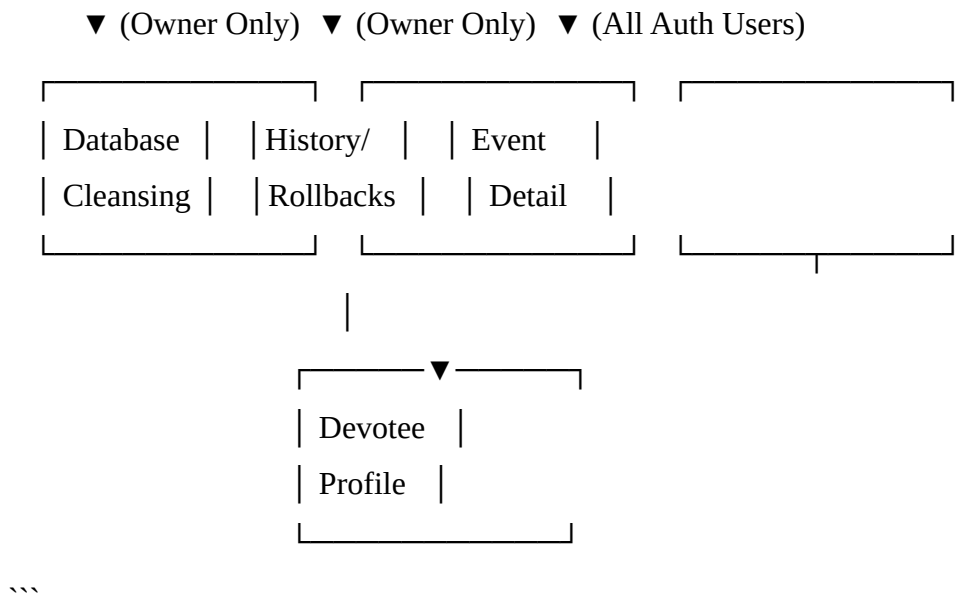
The system is compiled using **React 18** with **Vite** and backed by **Firebase (Firestore + Auth)**.

4.1 Client-Side Navigation Map (Routes)

- `/login`` - Entry point for authentication.
- `/`` - Main Dashboard (dynamically shows User Dashboard, Mentor Dashboard, or Owner Suite depending on user role).
- `/events/:id`` - Complete event detail pane (lists assignments, response breakdowns, custom charts).
- `/database`` - Admin Devotee Directory & Cleansing Suite (Owner-only view with soft deletes and merge utilities).
- `/attendance`` - Attendance control terminal (tracking live counts, open/close states, and template adjustments).
- `/history`` - Reversible Audit logs tracking database mutations.
- `/profile/:id`` - Detailed devotee dashboard containing history, logs, chanting rounds, and contact links.
- `/public-attendance/:id`` - Public attendance registration form for contactless sign-ins.

...





4.2 Data Persistence Blueprint

The following representation details the structured Firestore Schemas stored inside database documents:

1. `./users/{userId}` (UserProfile)

```

```json
{
 "displayName": "Aravind Sharma",
 "email": "aravind@temple.org",
 "role": "MENTOR",
 "contact": "+919988776655",
 "templeId": "temple_main_01",
 "mentorId": "owner_sri_01"
}
```

```

2. `./devotees/{devoteeId}` (Devotee Record)

```

```json
{
 "name": "Amit Kumar Patel",

```

```
"contact": "9812345678",
"mentor": "Sundeepr Prabhu",
"chanting": "16",
"facilitatorId": "facilitator_04",
"facilitatorName": "Rahul Das",
"attendanceCount": 12,
"createdAt": "2026-06-05T14:32:00Z",
"isDeleted": false,
"isDuplicate": false,
"duplicateType": null
}
...
```

#### 3. `/events/{eventId}` (Event Document)

```
```json
{
  "title": "Sri Gaura Purnima Festival",
  "date": "2026-06-15T18:00:00Z",
  "description": "Grand bathing ceremony and discourse.",
  "mediaUrl": "https://images.unsplash.com/photo-festival",
  "isPublic": true,
  "isAttendanceOpen": true,
  "createdBy": "owner_sri_01",
  "createdAt": "2026-06-06T09:00:00Z",
  "templateId": "form_festival_standard",
  "isDeleted": false
}
...
```

4. `/events/{eventId}/assignments/{assignmentId}` (Calling Assignment)

```
```json
{
 "devoteeId": "devotee_amit_99",
```

```
"userId": "facilitator_04",
"status": "COMPLETED",
"response": "COMING",
"responseText": "Informed he will attend with three of his friends.",
"devoteeName": "Amit Kumar Patel",
"devoteeContact": "9812345678",
"updatedAt": "2026-06-06T09:45:00Z"
}
...
```

#### 5. `/events/{eventId}/attendance/{devoteeId}` (Attendance)

```
```json
{
  "devoteeId": "devotee_amit_99",
  "present": true,
  "markedAt": "2026-06-06T09:50:00Z",
  "markedBy": "owner_sri_01"
}
...
```

6. `/templates/{templateId}` (Attendance Template)

```
```json
{
 "name": "Youth Festival Template",
 "fields": ["name", "contact", "institute", "age", "chanting"],
 "createdAt": "2026-06-01T10:00:00Z"
}
...
```

#### 7. `/temples/{templeId}/history/{historyId}` (Activity Logs)

```
```json
{
  "templeId": "temple_main_01",
```



```

"userId": "owner_sri_01",
"userName": "Sri Krishna Das",
"type": "DEVOTEE_EDIT",
"isUndone": false,
"createdAt": "2026-06-06T09:51:00Z"
}
...

---
```

5. Security & Isolation Invariants

To keep the system secure against active script injectors, privilege escalation, and direct data scraping, the application enforces rigid data constraints via `firestore.rules`.

5.1 Rules Compliance & Validation Rules

- **Role Alteration Isolation:** The field `role` inside any update payload directed at `/users/{userId}` is strictly immutable by the standard authenticated user. Only authenticated owners (`request.auth.uid` referencing an account where `resource.data.role == "OWNER"`) can modify role mappings.
- **Caller Data Bounds:** A user with the role of `USER` can strictly modify only the calling assignments assigned directly to them (checked via `resource.data.userId == request.auth.uid`).
- **Devotee Creation Safety:** Users of any role are allowed to append records to the devotee collection, safeguarding mobile lead ingestion, but deletions are exclusively permitted for the `OWNER`.
- **Public Registration Isolation:** The public registration page can append attendance logs inside `/events/{eventId}/attendance/{devoteeId}` ONLY when the parent event has the flag `isPublic == true` and `isAttendanceOpen == true`. Writing to events with locked headers results in immediate rejection.

5.2 Protection Against the "Dirty Dozen" Vulnerabilities

The codebase is hardened to fail-safe against the following attack scenarios with explicit ``PERMISSION_DENIED`` assertions:

1. ****Privilege Escalation:**** Injector attempts to write ``role: "OWNER"`` on an existing ``USER`` profile. (Blocked by Firestore immutability check on user fields).
2. ****Unauthorized Event Creation:**** A regular ``USER`` attempts to post a custom event payload. (Blocked; rules strictly require administrative claims for writing event documents).
3. ****Identity Spoofing in Assignments:**** Call Executor A attempts to change Call Executor B's calling assignment status. (Blocked; compared via ``request.auth.uid``).
4. ****Illegal Field Injection:**** Tampering with a ``Devotee`` profile by injecting unchecked, unverified database keys. (Blocked by checking validation rules against blueprint constraints).
5. ****Direct Attendance Hijacking:**** A student devotee attempts to inject ``present: true`` status for himself or others directly into the live event attendance stream. (Blocked unless event is explicit, open, and public).
6. ****Data Scraping (Blanket Database Dump):**** Requesting all global devotee contacts without an active auth signature. (Blocked; collection default is read-deny unless logged in).
7. ****Bypassing Assigned Calling List:**** A standard user attempts to download assignments not registered to their ``userId``. (Blocked; query filter bounds must explicitly match ``request.auth.uid``).
8. ****Shadow Moderator Mutation:**** A ``MENTOR`` attempting to hard-delete devotee profiles from the directory index. (Blocked; strict role validation restricts delete verbs to owners).
9. ****Tampering with Terminal State:**** Attempting to update calling responses or statuses after an event is finalized and locked. (Blocked via rules validating active dates).
10. ****Resource Exhaustion Attacks:**** Attempting to crash the sync thread by sending massive, oversized strings (e.g., a 1MB devotee name). (Validated in client-side text-limits and rule-length checks).
11. ****Orphaned Database Actions:**** Attempting to register assignments for an event ID that does not exist in the collection. (Cleaned up by cascade verification constraints).
12. ****PII Leakage Prevention:**** Standard callers searching for devotee contact detail keys that do not match their assigned cohort. (Constrained through mentor structure and security rules).

6. Performance, Escalation & Scaling Rules

6.1 Solutions for Client-Side Filtering Latency

- **The Issue:** Traditional code paths that execute `filteredDevotees = devotees.filter(...)` face O(N) performance degradation where client machines freeze when records approach 1,000+ devotees.
- **The Remedy:**
 1. **Memoized Query Filters:** Use optimized `useMemo` hooks with strict typing dependencies to lock UI updates.
 2. **Server-Side Firestore Query Indexing:** Offloads searching to Firestore engines using lightweight index filters, loading chunk structures on-demand.
 3. **Virtual Windowing:** For massive listing, client views render list cards virtualized, loading only viewable cards (viewport bounded).

6.2 Batched Transactions on Attendance Openings

- **The Issue:** Simultaneous marking/finalization of hundreds of devotees at a festival risks hitting Firestore write quota limits.
- **The Remedy:**
 1. **Firestore Batch Operations (`writeBatch`):** System collects updates into local batches and triggers bulk updates (in batches of 500 records) to stay safely within operational limits.
 2. **Throttling Updates:** Client queues calling updates and pushes items with localized debounce routines, preventing double clicking from triggering overlapping network streams.

6.3 Event Overload Prevention

- **The Issue:** Active events pulling massive sub-collection objects recursively can trigger data overhead.
- **The Remedy:**
 - Standard dashboard loads strictly current, open events. Historical data or massive assignments metadata are requested lazily only when transitioning to deep routes (`/events/:id`).

7. Product Acceptance Criteria

1. ****Security Compliance:**** Passing all mock firestore rule runs with rigid error handling assertions.
2. ****Visual Continuity:**** Immersive temple theme adhering to off-white, golden-orange and slate gray palettes across responsive desktop and mobile viewports.
3. ****Data Protection:**** Standard duplicate indicators triggered perfectly upon entry of overlapping name/contact structures.
4. ****Reversibility:**** 100% confirmation of the integrity of rollback actions executed in the Audit historical timeline.